ACTI 4850 – Lab 6 – Shared Pipeline Libraries

Instructor	Mike Mulder (mmulder10@bcit.ca)
Total Marks	10
Due Dates	Demo Due by End of Next Class: • Feb. 17th for Sets B and C • Feb. 18th for Set A

Applicable Requirements

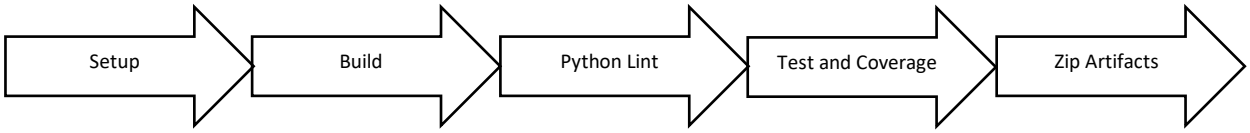
- **REQ1150 –** The Enterprise Development Environment shall automatically trigger a Continuous Integration Pipeline on a Software Project (i.e., repository) in Source Code Management when the source code changes. Note: A push to a project in GitLab should trigger the corresponding build job.
- **REQ1160 –** The Enterprise Development Environment shall support Continuous Integration Pipelines for Python projects. At minimum, the pipeline will include build, test, packaging and artifact storage. Note the Python steps in the Build and Test stages must run in a Virtual Environment.
- **REQ1170** – The Enterprise Development Environment shall support re-use of CI Pipeline definitions across similar projects. For example, similar Python projects (i.e., Flask applications) should be able to use the same pipeline definition.

Group Work

You will be working on the same Azure cloud environment as the previous lab, shared with your Lab partner. This lab will be done together with your partner.

<u>Part 1 – Extend Your Point Project's Pipeline</u>

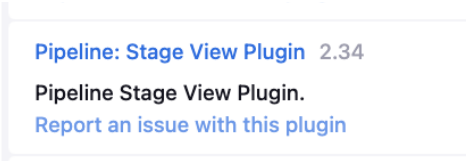
Add (or modify) the following stages to the Jenkinsfile for you Point Project (in the given order):



Note: You can use the Jenkins Replay feature to test your changes rather than making a large number of Git commits.

Prerequisite

To get the Pipeline View in your Jenkins Pipelines, install this plugin in your Jenkins installation (Manage Jenkins -> Plugins -> Available Plugins:



This plugin will give show you the status of each pipeline stage for each build:

Declarative: Checkout SCM	Build	Python Lint	Test and Coverage	Zip Artifacts
948ms	3s	1min 38s	44s	318ms
1s	3s	428ms	3s	499ms

This plugin used to be installed by default in Jenkins, but seems to require a manual install now.

Setup – New Stage

- Note that you will need to install the following in your Jenkins Agent docker image (i.e., update the dockerfile then stop/remove/build and start the image):

  apt-get install –y python3.13-venv

  Note: You should be able to add this to the existing apt-get install line in the dockerfile.
- Create this new stage that:
  - Displays the location of job's workspace using the WORKSPACE environment variable in Jenkins. You can use an echo command in the steps of your stage, something like:

    ```
    echo "Running ${env.BUILD_ID} with workspace ${env.WORKSPACE}"
    ```

  - Cleans up the current workspace. We will use the fileExists, dir and deleteDir Groovy functions here to cleanup the test-reports, api-test-reports and venv folders if they exist. Since this is Groovy code, it needs to be wrapped in a script block. For example:

    ```
    script {
        def foldersToDelete = ['test-reports',
                               'api-test-reports',
                               'venv']

        for (folder in foldersToDelete) {
            if (fileExists(folder)) {
                dir(folder) {
                    deleteDir()
                }
                echo "Deleted directory: ${folder}"
            }
            else {
                echo "Directory not found, skipping: ${folder}"
            }
        }
    }
    ```

- Display the contents of the three folders to show that they are empty (i.e., you can use ls commands)
- Create a Python Virtual Environment (using a venv folder) in the new clean workspace. For example:

  python3 -m venv venv

  Ref: https://docs.python.org/3/library/venv.html

Build – Existing Stage

- Leave as is, but make sure you pip install command from your Virtual Environment, i.e., venv/bin/pip install -r requirements.txt
  The --break-system-packages is no longer needed here.

Python Lint – New Stage

- Runs Python Lint
- Note that you will need to install the following in your Jenkins Agent docker image (i.e., update the dockerfile then stop/remove/build and start the image):

  apt-get install –y pylint

  Note: You should be able to add this to the existing apt-get install line in the dockerfile.

- You will need to run pylint --fail-under <Score> <Files> in your Lint Stage where the app will fail with a lint score under 5 and all .py files are scanned. See https://pylint.readthedocs.io/en/latest/user_guide/configuration/all-options.html#fail-under
- You can use wildcards to apply this to all .py files

Test and Coverage – Rename of Unit Test Stage

- Because we want to run coverage in our virtual environment, we'll need to install it in that environment. That can be part of your Setup stage and done right after the Virtual environment is created:

  venv\bin\pip install coverage

  We don't want to add this to our requirements.txt since it is only used in the Pipeline and is not part of the application itself.

  Ref: https://coverage.readthedocs.io/en/coverage-5.0.3/

- Modify this stage so it runs each file in the workspace that starts with "test" as a unittest
  - You will need to install the Pipeline Utilities Steps plugin in Jenkins to get the ability to get a filtered list of all files in a workspace (search on findFiles)

    - Use a for loop (as in the slides) to run each test file using coverage (see hints below).
- Have a Post block that processes both the test-results and api-test-results data using some Groovy code in a script block:

```
// Process the test results if they exist
script {
      def test_reports_exist = fileExists 'test-reports'
      if (test_reports_exist) {
          junit 'test-reports/*.xml'
      }

      def api_test_reports_exist = fileExists 'api-test-reports'
      if (api_test_reports_exist) {
          junit 'api-test-reports/*.xml'
      }
}
```

- Hints on using coverage (**see the slides for Week 6 on the pattern to use**):

  For each test file run:
  coverage run --omit */site-packages/*,*/dist-packages/* <test file name>
  The above should replace your test runs as it will run the tests and generate the test results as well as generating the coverage (otherwise you will get double the number of test results).

  After all the test are run (i.e., after your for loop):
  coverage report

Integration Test – Remove this Stage

- Remove this stage altogether. It is replaced by the Test and Coverage Stage above, which will run all Python tests.

Zip Artifacts – New Stage

- Zip up all the Python files (i.e., all files with a .py extension) into a zipfile called app.zip
    - You'll need to install zip on your Jenkins docker image
- Make that available as an artifact in the Jenkins build job for download
    - Look up the archiveArtifacts step in Jenkins

Your pipeline in Jenkins should look like this when successfully run:

Declarative: Checkout SCM	Build	Python Lint	Test and Coverage	Zip Artifacts
948ms	3s	1min 38s	44s	318ms
1s	3s	428ms	3s	499ms

Commit your changes to your Git project.

<u>Part 2 – Create a Shared Library for Your Point Project</u>

Create a new Project in GitLab called “ci_functions”.

Create a subdirectory called “vars”.

Create a file called python_build.groovy inside the vars directory.

Put your updated pipeline from Part 1 into the python_build.groovy file and make it callable.

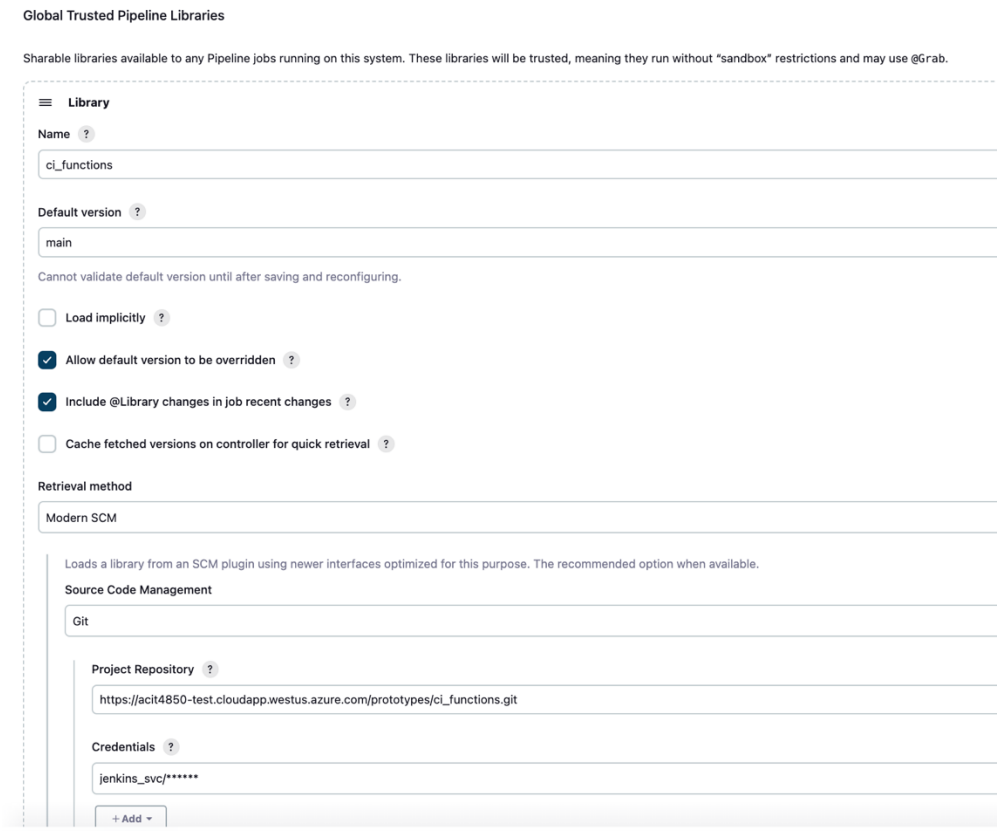
```
def call() {

    // Your Pipeline Here

}
```

Commit the code to the ci_functions project.

Update your Jenkins installation (i.e., Manage Jenkins -> Configure Jenkins -> Global Trusted Pipeline Libraries) so that ci_functions is a Global Trusted Pipeline Library. (on the main branch).



Update the Jenkinsfile in Point project to call the pipeline in python_build.groovy (see the example in the Week 6 slides).

Test and make sure the Point project still builds.

Part 3 – Add a New Python Project Using the Same Pipeline Shared Library

Download the CarLotSample.zip file from the Week 6 content on D2L.

Create a new project in GitLab called CarLot and push the code into the repo.

- Add a requirements.txt file with specific versions of Flask, SQLAlchemy, Xmlrunner and requests (use the requirements.txt in the point repo as an example). Push this file to the repo.
- Add a ci/Jenkinsfile that includes your ci_functions library and calls the pipeline in python_build.groovy.
- You will need to update the unit test to generate xml based unit test results (see test_point_manager.py from the point repo as an example).

Create a new CarLotPipeline pipeline build job in Jenkins for your new CarLot project that uses the ci/Jenkinsfile from the repo. Test out this build job and make sure it builds successfully for all the stages (there likely will be issues you need to fix). Fix any issues.
Make sure to setup a Webhook for this repo/job as well so it automatically builds on a push to the repo.

<u>Make sure you shutdown any Azure resources (i.e., the VM) to conserve your credits and free tier usage.</u>

Demo, Grading and Submission

A demo of your lab against the applicable requirements which will determine your grade on the lab. All mandatory requirements must be met otherwise you will receive zero on the lab. You can re-demo the lab if you haven't met the mandatory requirements, up to the last class before the midterm week, but you will lose 20% every week late.

Reqt.	Mandatory	Demo	Marks
REQ1150	Yes	• PointPipeline is triggered by pushes to the corresponding GitLab repo. • CarLotPipeline is triggered by pushes to the corresponding GitLab repo.	2

REQ1160	Yes	• PointPipeline – Runs on the Python Agent
		• All five pipeline stages for PointPipeline:
		  ○ Setup
		  ○ Build
		  ○ Lint
		  ○ Test (including Coverage)
		  ○ Package
		• **The Python steps are run in a Virtual Environment**
		• The app.zip file is an artifact
		• Uses the python_build.groovy shared library
		• Shows the correct output in the Test graph
			4

REQ1170	Yes	• CarLotPipeline – Runs on the Python Agent
		• All five pipeline stages for PointPipeline:
		  ○ Setup
		  ○ Build
		  ○ Lint
		  ○ Test (including Coverage)
		  ○ Package
		• **The Python steps are run in a Virtual Environment**
		• The app.zip file is an artifact
		• Uses the python_build.groovy shared library
		• Shows the correct output in the Test graph
			4

Total			**10**

<u>Submit your python_build.groovy file to the Lab 6 dropbox on D2L.</u>